COS30018 - Option C - Task 5 Report

Machine Learning 1

Student Name: Kha Anh Nguyen
Student ID: 103814796
Date: September 5, 2025
Course: COS30018 - Intelligent Systems
Project: FinTech101 Stock Price Prediction System
Github repo: Leissha/stock_prediction

Table of Contents

I did not code much this week since I already allowed lookup step (prediction days) parameter, and multivariate inputs in the previous labs. Detailed code is mentioned in Task 2 report for data preprocessing decisions. This report focuses on the multistep and multivariate prediction implementations.

1. Key changes:

- I updated some functions to accept the future prediction steps' array.
- I updated the **create_sequences** function, **DataProcessor**, **TFModel** class and **train** function to accept the "lookup_steps" parameter through the entire pipeline for multistep prediction support.
  (I set the lookup_steps to be interger = 1 as default before).
- In addition, I refactor & separate the main training code file into 3 files, each with distinct functionalities:
    1. main.py for CLI arguments parsing and orchestrate the whole pipeline
    2. train.py to train models
    3. test.py to predict and evaluate the trained model.

2. Multistep predictions

Create sequence

- I add a functionality block to turn the integer prediction day to an array of prediction days integer.

```
 # Future target: create sequence of future values (unified logic)

future_sequence = []
for step in range(lookup_steps):
    future_sequence.append(scaled_data[i + step, target_indices])

# Always return as array for consistency
y.append(future_sequence)
```

TFModel Base Model:

The base model also store the prediction days length attribute.

```
def __init__(self, input_size: int, model_name: str = MODEL_NAME, layers:
Optional[List[int]] = LAYERS,
             dropout_rate: float = DROPOUT, output_steps: int = 1):
    # ...
    self.output_steps = int(output_steps)
```

So the build model output layer can be flexible:

```
def _build_model(self):
    # ...
    # Final prediction layer
    # When output_steps=1: units=1 (single prediction)
    # When output_steps>1: units=output_steps (multiple predictions)
    model.add(Dense(units=self.output_steps, activation='linear'))
```

Multivariate Prediction Recap

The input features accept 5 values, tickers at: Close, High, Low, Open and ticker's Volume

```
# Input features (OHLCV): ['close', 'high', 'low', 'open', 'volume']
training_features = ['close', 'high', 'low', 'open', 'volume']
```

I also use separated scaler for each feature at data preprocessing:

```
            for i, feature in enumerate(training_features):
                # Create and store individual scaler for each feature
                scaler_key = f"{ticker}_{feature}"
                self.scalers[scaler_key] = StandardScaler()

                # Fit scaler only on training data to prevent data leakage
                train_scaled[:, i] = self.scalers[scaler_key].fit_transform(
                    train_df[[feature]].values
                ).reshape(-1)
                test_scaled[:, i] = self.scalers[scaler_key].transform(
                    test_df[[feature]].values
                ).reshape(-1)
            # Cache all feature scalers for future inference use
            scaler_bundle = {
                "scalers": self.scalers,     # per-feature scalers dict
                "training_features": training_features,      # column order
                "target_scaler_key": f"{ticker}_{target_feature}",  # just the
key string
            }
```

3. Results

CLI

Multivariate inputs are already integrated in the pipeline, we can use --lookup_steps arg to make different prediction days:

```
### Combined Multivariate-Multistep
python main.py --company CBA.AX --lookup_steps 3 --epochs 50
```

Test Result Plots

Noted that I only use true values for lookback step to reduce the error accumulation.

LSTM model with lookup-steps = 1, 5, 10 and price/percentage return:

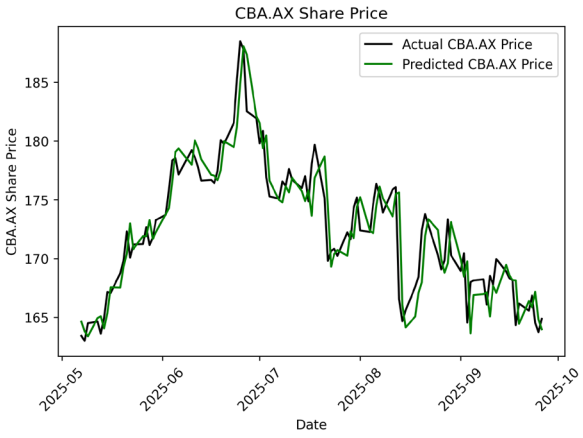

Figure 1. Lookup_steps = 1 with percentage-change target return

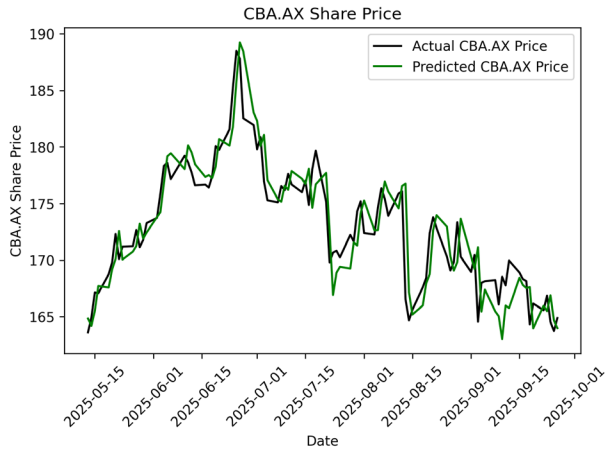

Figure 2 Lookup_steps = 5 with percentage-change target return

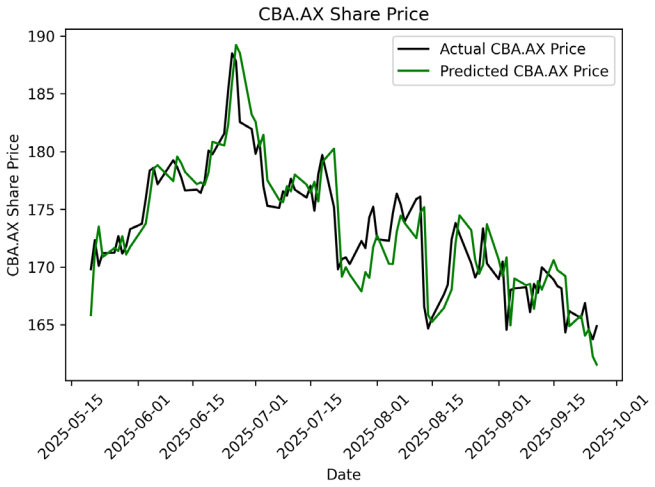

Figure 3 Lookup_steps = 10 with percentage-change target return

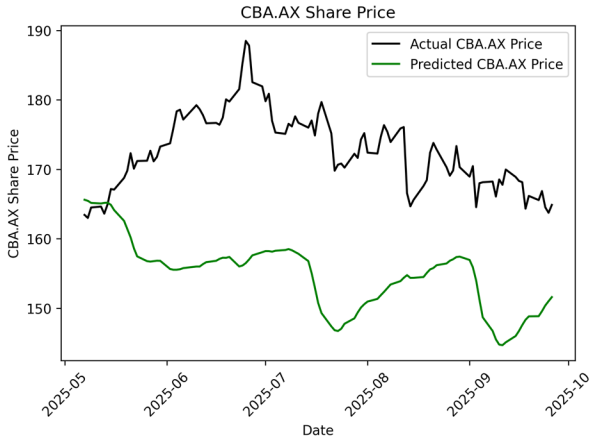

Figure 4 Lookup_step = 1 with normal price retur

BiLSTM misses the small changes as the predicted days increase from 5 to 10:

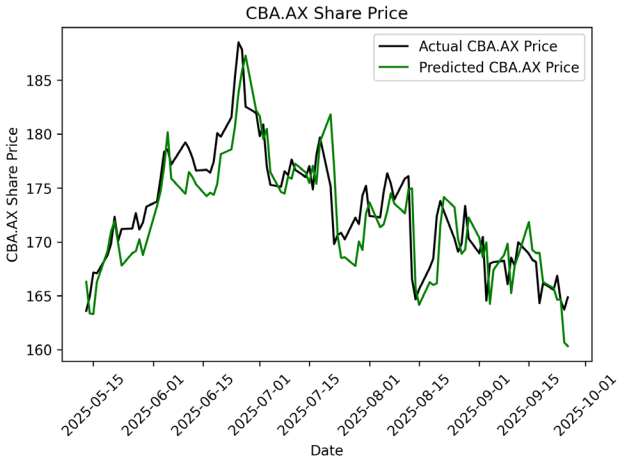

Figure 5 Lookup_steps = 5 with percentage-change target return

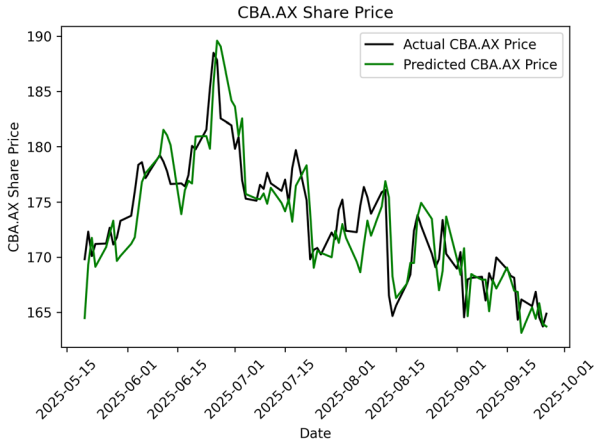

Figure 6 Lookup_steps = 10 with percentage-change target return

Statistic summary

The table below shows error accumulation is exponential: 50% increase at 5 days, 287% at 10 days

(I'm using percentage change price target so the loss is larger than original price target).

Model	Prediction Steps	MAE	RMSE	Directional Accuracy	Trading Accuracy	Total Profit	Profitable Trades
LSTM	1 day	0.420	0.420	55.79%	49.47%	$78.77	47/95
LSTM	5 days	0.630	0.630	50.59%	45.88%	$60.49	39/85
LSTM	10 days	1.625	1.625	48.86%	42.05%	$70.20	37/88
BiLSTM	5 days	2.088	2.088	48.42%	41.05%	$63.34	39/95
BiLSTM	10 days	0.270	0.270	48.35%	42.86%	$68.15	39/91
GRU	10 days	0.749	0.749	48.24%	40.00%	$54.35	34/85

---

4. Conclusion

The implementation demonstrates the fundamental challenge of multistep time series forecasting, where prediction accuracy degrades exponentially with increasing prediction horizons (50% error increase at 5 days, 287% at 10 days). While the unified architecture effectively handles both single-step and multistep predictions using multivariate OHLCV inputs, the results confirm that practical trading applications should favor shorter prediction windows (1-2 days) for reliable risk management. The exponential error accumulation pattern observed across all model architectures (LSTM, BiLSTM, GRU) validates the theoretical expectation that financial time series prediction becomes increasingly unreliable beyond short-term horizons.